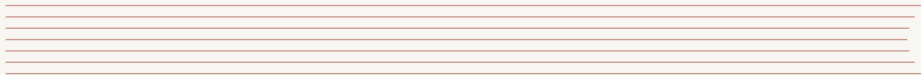


OPTIMISATION CONJOINTE DES BLOCS OPÉRATOIRES ET DES LITS D'HOSPITALISATION

ÉTAT DE L'ART, MODÉLISATION ET MÉTAHEURISTIQUES

SOUTENANCE INTERMÉDIAIRE – FIL ROUGE
PROJET ALTAO

14 SEPTEMBRE 2026



- **État de l'art**
- **Modélisation**
- **Métaheuristiques**
- **Suite du travail**

SECTION 1

État de l'art

Trois niveaux de décision, largement étudiés séparément :

Niveau	Horizon	Décisions
Stratégique	1–5 ans	Dimensionnement des salles
Tactique	1–3 mois	Programme maître de chirurgie (MSS)
Opérationnel	jour – semaines	Affectation salle / patient / lit

I Formulation dominante

Programmation linéaire (mixte) : variables d'affectation patient → bloc → lit, sous contraintes de capacité et de durées, fonction objectif minimisant retards et dépassements.

Le problème d'ordonnancement bloc/lits est **NP-difficile** : la programmation linéaire exacte devient vite trop coûteuse en calcul. Deux familles de méthodes approchées coexistent.

I Heuristique

- Règle empirique conçue pour **un problème précis**
- Rapide, peu coûteuse en calcul
- Aucune garantie d'optimalité : risque d'optimum local

I Métaheuristique

- Cadre **générique**, applicable à des problèmes NP-difficiles variés
- Explore plus largement l'espace des solutions
- Meilleure qualité de solution, au prix d'un calcul plus long

Limite principale

Blocs opératoires et lits d'hospitalisation sont **le plus souvent optimisés séparément** : le bloc est planifié, puis le lit est recherché a posteriori.

- Peu de modèles **conjointes** bloc + lit dans les approches classiques (programmation linéaire).
- Les incertitudes (urgences, variabilité des durées) sont rarement intégrées de façon robuste dans les modèles conjoints.

⇒ **Optimisation conjointe blocs + lits**

SECTION 2

Modélisation

VARIABLES (1/2) : ENSEMBLES

Notation	Description
$P = \{1, \dots, n\}$	Ensemble des patients à planifier
$S = \{1, \dots, m\}$	Ensemble des salles opératoires
$T = \{1, \dots, H\}$	Jours sur lesquels il faut planifier
$C = \{c_1, \dots, c_k\}$	Ensemble des chirurgiens
$V_{c,s,t}$	= 1 si le chirurgien c a une vacation dans la salle s le jour t , 0 sinon

VARIABLES (2/2) : PARAMÈTRES

Notation	Description	Origine
d_i	Durée opératoire prédite du patient i	BDD (heures bloc)
S_i	Durée de séjour prédite (en jours)	BDD (durée séjour)
a_i	= 1 si ambulatoire, 0 sinon	BDD (séjour = 1j)
C_{vac}	Capacité d'une vacation	—
C_{max}	Capacité temporelle totale du bloc/jour	—
L_{max}	Capacité maximale en lits	Clinique
A_{max}	Capacité maximale en places ambu.	Clinique
$L_{\text{cible}}(t)$	Occupation cible des lits au jour t	± 2 lits
$A_{\text{cible}}(J)$	Occupation cible des places ambu.	Hôpital
$\tau_{\text{Urgence}}(J)$	Temps réservé urgences (bloc)	BDD
$L_{\text{Urgence}}(J)$	Lits réservés urgences	BDD
$A_{\text{Urgence}}(J)$	Places ambu. réservées urgences	BDD
$x_{i,s,t} \in \{0, 1\}$	= 1 si patient i opéré salle s jour t	Variable
$L(\tau)$	Occupation lits classiques jour τ	Calculé
$A(\tau)$	Occupation places ambu. jour τ	Calculé

CONTRAINTES (1/2)

C1 : Chaque patient correspond à une seule opération :

$$\sum_{s \in S} \sum_{t \in T} x_{i,s,t} = 1 \quad \forall i \in P$$

C2 : Capacité de vacation (marge urgences) :

On réserve une marge temporelle pour les urgences chirurgicales du jour.

$$\sum_{i \in P} d_i \cdot x_{i,s,t} \leq C_{\text{vac}} - \tau_{\text{Urgence}}(t) \quad \forall s \in S, \forall t \in T$$

C3 : Planning des chirurgiens :

On ne peut planifier des interventions sur des créneaux non libres.

$$x_{i,s,t} \leq V_{\text{chir}_i,s,t} \quad \forall i \in P, s \in S, t \in T$$

C4 : Capacité en lits (marge urgences) :

On réserve des lits pour les urgences du jour.

$$L(\tau) \leq L_{\max} - L_{\text{Urgence}}(\tau) \quad \forall \tau \in T$$

C5 : Capacité en places ambulatoires (marge urgences) :

On réserve des places pour les urgences ambulatoires du jour.

$$A(\tau) = \sum_{\substack{i \in P \\ a_i=1}} \sum_{s \in S} x_{i,s,\tau} \leq A_{\max} - A_{\text{Urgence}}(\tau) \quad \forall \tau \in T$$

Minimiser les fluctuations d'occupation et optimiser l'utilisation du bloc :

$$\begin{aligned} \min \quad & \underbrace{\alpha \sum_{J \in T} (L(J) - L_{\text{cible}}(J))^2}_{\text{Lissage lits}} + \underbrace{\beta \sum_{J \in T} (A(J) - A_{\text{cible}}(J))^2}_{\text{Lissage ambulatoire}} \\ & + \underbrace{\gamma \sum_{J \in T} \left(\sum_{i \in P} \sum_{s \in S} d_i x_{i,s,J} - [C_{\text{max}} - \tau_{\text{Urgence}}(J)] \right)^2}_{\text{Utilisation du bloc (marge urgences)}} \end{aligned}$$

- Terme 1 : pénalise les **pics et creux** d'occupation des lits.
- Terme 2 : lisse la charge des **places ambulatoires**.
- Terme 3 : pénalise l'écart entre la capacité bloc (urgences comprises) et la capacité réelle.
- $\alpha, \beta, \gamma > 0$: poids de pondération à ajuster.

SECTION 3

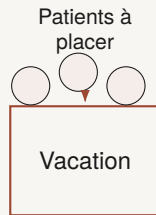
Métaheuristiques

RAPPEL : UN PROBLÈME D'AFFECTATION COMBINATOIRE

I Ce qu'il faut décider chaque jour

Affecter des **patients** à des **vacations de bloc** (créneaux de 4h) et des **lits**, sous contraintes :

- Durée opératoire prédite par patient
- Capacité de chaque vacation / spécialité
- Compatibilité chirurgien – créneau
- Disponibilité des lits en aval
- Marge à garder pour les urgences



Le point clé

Chaque combinaison possible est une solution candidate – et leur nombre explose avec le nombre de patients et de créneaux.

POURQUOI UNE MÉTHODE EXACTE NE SUFFIT PAS

I Un problème NP-difficile

Ce type d'affectation sous contraintes multiples appartient à la famille des problèmes de **scheduling combinatoire** : le nombre de combinaisons croît de façon exponentielle avec le nombre de patients et de ressources.

I Approche exacte

- Garantit l'optimum
- Temps de calcul explose vite
- Inutilisable à l'échelle d'un hôpital, en temps utile

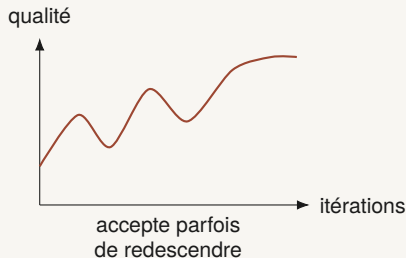
I Métaheuristique

- Pas de garantie d'optimum absolu
- Très bonne solution en temps raisonnable
- S'adapte à des contraintes complexes et changeantes

⇒ *on échange une garantie théorique contre une décision exploitable chaque jour.*

Principe

- Part d'une solution (un planning) et la modifie légèrement
- Accepte parfois une solution *moins bonne*, avec une probabilité qui diminue au fil du temps (température)
- Évite de rester bloqué sur une première solution correcte mais médiocre

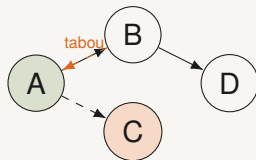


■ Pourquoi ce choix ici

Utile en phase de **construction initiale** d'un planning de vacances : il balaie largement l'espace des combinaisons possibles sans s'enfermer trop vite dans une solution locale.

Principe

- Explore les plannings voisins du planning courant
- Choisit le meilleur voisin, même s'il n'améliore pas tout de suite
- Mémoire une **liste taboue** des derniers mouvements pour interdire d'y revenir

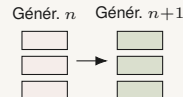


■ Pourquoi ce choix ici

Utile pour **raffiner** un planning déjà construit : ajuster finement l'affectation patient/créneau sans reboucler sur les échanges déjà testés.

Principe

- Une **population** de plannings évolue en parallèle
- Croisement : combine deux bons plannings
- Mutation : introduit de petites variations
- Sélection : garde les plus performants



Méthode	Rôle dans le projet	Force
Recuit simulé	Construction initiale	Large exploration
Tabou	Raffinement local	Évite les boucles
Génétique	Recherche de compromis global	Diversité de solutions

À retenir

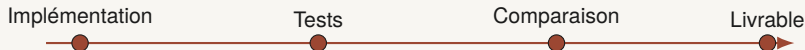
Trois logiques complémentaires – pas rivales – pour transformer un problème NP-difficile en un planning exploitable chaque jour.

SECTION 4

Suite du travail

I Trois chantiers pour la suite du projet

- 1 **Implémentation des algorithmes** (recuit simulé, tabou, génétique) en Python.
- 2 **Tests sur la base de données** réelle de la clinique.
- 3 **Comparaison des performances** des trois métaheuristiques.

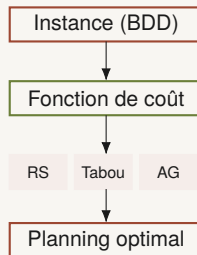


Objectif

Passer des modèles mathématiques à un **prototype exécutable et mesuré**, utilisable comme base de décision.

Un moteur Python modulaire

- Représentation commune d'un **planning** (solution)
- Fonction de coût reprenant l'objectif (lissage lits / ambu. / bloc)
- Un module par méthode : `recuit.py`, `tabou.py`, `genetique.py`
- Même interface `solve(instance)` -> `planning` pour les comparer à conditions égales



I Données utilisées

- Historique d'interventions de la clinique
- Durées opératoires et de séjour
- Part d'ambulatoire, vacations des chirurgiens

I Niveaux de test

- **Unitaires** : contraintes C1–C5 respectées
- **Intégration** : lecture BDD → planning
- **Validation** : planning faisable et exploitable

Jeux de tests

Une **instance jouet** (quelques patients) pour déboguer, puis des **instances réelles** de taille croissante pour éprouver le passage à l'échelle.

I Traitement – Python / FastAPI

- Moteur d'optimisation en **Python**
- **FastAPI** expose les algorithmes via une API REST
- Validation des données par **Pydantic**
- Calcul scientifique : **NumPy / Pandas**

I Interface web – React

- **React** (Vite) pour un tableau de bord interactif
- Visualisation des plannings et des performances
- Appels à l'API FastAPI
- Comparaison des méthodes en un coup d'œil

Pourquoi ce choix

FastAPI est rapide, typé et documenté automatiquement (Swagger) ; React offre une interface réactive et portable, séparant clairement le **moteur de calcul** de la **présentation**.

RÉFÉRENCES

Merci de votre attention

Questions ?